

Contents

01 - Getting Started	3
Installation	3
Project Initialization	3
Your First Program	4
Writing Comments	4
02 - Variables and Types	4
Variable Declaration	4
Immutable Variables with @const	4
Type Annotations	5
Basic Types	5
String Operations	6
Escape Sequences	7
F-String (String Interpolation)	7
Type Casting (as)	7
Reassignment Rules	8
Tuple Destructuring	8
03 - Operators	9
Arithmetic Operators	9
Comparison Operators	9
Logical Operators	10
Bitwise Operators	10
Compound Assignment Operators	11
Increment / Decrement Operators	11
Type Promotion Rules	11
Membership Operators	12
Control Flow	12
if / else	12
case	13
while Loop	14
for Loop and range	14
break and continue	15
Nesting Example	16
Scope Rules	16
Pattern Matching	17
Functions	18
Basic Function Definition	18
Calling Functions	18
Recursive Functions	18
Function Overloading	18
Omitting the Return Type (Unit Type)	19
Default Arguments	19
Lambda Functions	19
Closures	20
Higher-Order Functions	21
UFCS (Uniform Function Call Syntax)	21
Exercises	22
Records and Enums	22
Defining Records	22

Creating Instances	22
Field Access (Dot Notation)	22
Field Assignment	23
Records as Function Parameters	23
Nested Records	23
Enums	23
Enum with Associated Data (ADT)	24
Generic Enums	25
Operator Overloading	25
Exercises	26
Collections and Iterators	27
Tuples	27
Lists	27
Maps	30
Sets	31
Iterators	31
Exercises	33
Error Handling	33
Option Type	33
Result Type	34
Design by Contract	35
When to Use What	37
Common Mistakes	37
Exercises	37
Packages	38
from/import Syntax	38
Subdirectories (Dot-Separated Paths)	38
Directory Packages	38
Relative Imports	38
Standard Library (std)	39
Search Path Priority	39
RY_PATH Environment Variable	39
Limitations	39
Concurrency	40
async/await	40
@parallel	41
OS Threads	41
Networking (TCP Sockets)	43
Choosing the Right Model	44
Common Mistakes	44
Exercises	45
Testing	45
Running Tests	45
Writing Tests	45
Matchers	46
Output Format	47
Mocking	47
Parameterized Tests	48
Property-Based Tests	48
Testing with Contracts	48

Limitations	49
Exercises	49
Building a Project	49
Project Setup	49
Step 1: Define the Data Model	50
Step 2: Task Operations	50
Step 3: Display	51
Step 4: Main Program	51
Step 5: Write Tests	52
Next Steps	53
English 日本語 繁體中文	

01 - Getting Started

Next tutorial -> [02 - Variables and Types](#)

Installation

Quick Install (macOS Apple Silicon)

```
curl -fsSL https://raw.githubusercontent.com/t0k0sh1/ry/main/install.sh | sh
```

This installs the ry binary to ~/.local/bin and the standard library to ~/.ry/share/std/.

Make sure ~/.local/bin is in your PATH:

```
export PATH="$HOME/.local/bin:$PATH"
```

To build from source or install on other platforms, see the [Installation section in README](#).

Project Initialization

You can create a new project with the `ry new` command:

```
ry new my-project
cd my-project
```

This generates the following files and directories:

- `package.toml` – Project configuration file
- `src/main.ry` – Entry point (with sample code)

To initialize the current directory as a project instead, use `ry init`:

```
mkdir my-project
cd my-project
ry init
```

See [Project Management](#) for details.

Your First Program

Save the following content to a file named `hello.ry`:

```
print("Hello, World!")
```

Run it with the following command:

```
ry hello.ry
```

Output:

```
Hello, World!
```

You can also run code from stdin using a pipe or here-document with the `-c` flag:

```
echo 'print("Hello, World!")' | ry -c
```

```
ry -c <<'RY'
print("Hello, World!")
RY
```

Writing Comments

Everything from `#` to the end of the line is treated as a comment.

```
# This is a comment
print("Hello") # End-of-line comments are also supported
```

Comments do not affect the behavior of the code.

Next tutorial -> [02 - Variables and Types](#)

[English](#) | [日本語](#) | [繁體中文](#)

02 - Variables and Types

<- [01 - Getting Started](#) / Next -> [03 - Operators](#)

Variable Declaration

In Ry, variables are declared using simple assignment syntax. By default, variables are mutable.

```
x = 42          # Inferred as int
y = 3.14        # Inferred as float
flag = true     # Inferred as bool
name = "Ry"     # Inferred as str
```

Immutable Variables with `@const`

The `@const` directive marks a variable as immutable (constant). The value cannot be changed after declaration.

```
@const
x = 42          # Inferred as int
```

```

@const
y = 3.14      # Inferred as float

@const
flag = true   # Inferred as bool

@const
name = "Ry"   # Inferred as str

```

Type Annotations

You can explicitly specify the type of a variable.

```

x: int = 42

rate: float = 0.5

ok: bool = false

msg: str = "hello"

```

A compile error occurs if the type annotation does not match the actual type of the value.

Basic Types

Type	Description	Literal Examples
int	64-bit integer	0, 42, -10
u8	Unsigned 8-bit integer (0-255)	b: u8 = 42
float	64-bit floating-point number	0.0, 3.14, -1.5
bool	Boolean	true, false
str	String	"hello", ""

Low-Level Numeric Types

Ry also provides low-level numeric types for precise control over memory layout. These types have **no implicit conversions** — you must use **as** for explicit casts.

Type	Description	Example
i8	8-bit signed integer	x: i8 = 42
i16	16-bit signed integer	x: i16 = 100
i32	32-bit signed integer	x: i32 = 42
i64	64-bit signed integer	x: i64 = 100
u8	8-bit unsigned integer	x: u8 = 200
u16	16-bit unsigned integer	x: u16 = 60000
u32	32-bit unsigned integer	x: u32 = 3000000000
u64	64-bit unsigned integer	x: u64 = 100
f32	32-bit floating-point	x: f32 = 3.14

```

a: i32 = 10
b: i32 = 20

```

```

c = a + b           # OK: i32 + i32 → i32

d = 42
# e = a + d         # Error: cannot mix i32 and int

y = a as int        # Explicit cast to int
z = d as i32         # Explicit cast to i32

# Unsigned types use unsigned operations
x: u32 = 3000000000
y: u32 = 7
q = x / y           # Unsigned division (UDiv)

```

Note: / on low-level integers performs integer division (like Rust), not float division. Signed types use SDiv, unsigned types use UDiv.

Note: Arithmetic on low-level integers wraps on overflow. Signed types use two's complement, unsigned types use modular arithmetic. Use the high-level `int` type (64-bit) if overflow is a concern.

Fixed-Length Arrays

For low-level types, Ry provides fixed-length contiguous arrays `T[N]`. These are stack-allocated with compile-time known size.

```

buf: i32[4] = [1, 2, 3, 4]
print(buf[0])      # 1
buf[2] = 99
print(buf[2])       # 99
print(length(buf))  # 4

pixels: u8[3] = [255, 128, 0]

```

String Operations

Various operations are available for strings.

```

a = "Hello"
b = "World"

# Concatenation
c = a + ", " + b  # "Hello, World"

# Comparison (lexicographic order)
print(a == b)     # false
print(a != b)     # true
print(a < b)      # true ("H" < "W")

# Length
print(length(a))  # 5

# Substring checks
s = "Hello, World!"
print(contains(s, "World"))  # true

```

```
print(starts_with(s, "Hello"))  # true
print(ends_with(s, "!"))       # true
```

Escape Sequences

The following escape sequences can be used within strings.

Sequence	Meaning
<code>\n</code>	Newline
<code>\r</code>	Carriage return
<code>\t</code>	Tab
<code>\\</code>	Backslash
<code>\"</code>	Double quote
<code>\0</code>	Null character

```
print("Hello\nWorld")  # Outputs on two lines
print("A\tB")          # Tab-separated
print("say \"hi\"")     # String containing double quotes
```

F-String (String Interpolation)

When you need to embed variables or expressions inside a string, use `f"..."` instead of concatenating with `+`. Expressions inside `{}` are evaluated and converted to strings automatically.

```
name = "Alice"
print(f"Hello {name}")  # Hello Alice

x = 3
y = 4
print(f"{x} + {y} = {x + y}")  # 3 + 4 = 7
```

Use `{{` and `}}` to include literal braces:

```
print(f"{{escaped}}")  # {escaped}
```

Why f-strings? They are more readable and less error-prone than manual concatenation like `"Hello " + name`. Use f-strings whenever you mix text with dynamic values.

Common mistake: Writing `"Hello {name}"` without the `f` prefix gives you the literal text `{name}`, not the variable's value.

Type Casting (as)

Convert between types explicitly with `as`. Narrowing conversions (like `float` to `int`) and conversions between numeric and non-numeric types (`str`, `bool`) require `as`, while some safe numeric promotions (such as `int` to `float` in expressions and certain calls) happen implicitly.

```
x = 42 as float      # 42.0
y = 3.14 as int       # 3 (truncated toward zero)
s = 42 as str         # "42"
b = true as int       # 1
```

Common mistake: Casting `float` to `int` truncates toward zero, so `3.9 as int` gives 3, not 4. Use `round()` from the `math` module if you need rounding.

Reassignment Rules

Variables declared without `@const` can be reassigned. However, the following restrictions apply:

```
x = 10
x = 20      # OK: reassignment to the same type
# x = "text" # Error: reassignment with a different type is not allowed
```

`@const` variables cannot be reassigned.

```
@const
N = 5
# N = 6 # Error: reassignment to a @const variable is not allowed
```

Redeclaring a variable with the same name is also not allowed.

```
x = 1
# x = 2 with another declaration in the same scope is not allowed
```

Tuple Destructuring

You can unpack a tuple into multiple variables in a single declaration.

```
@const
a, b = (10, 20)
print(a)    # 10
print(b)    # 20
```

Wildcard

Use `_` to ignore a position.

```
@const
x, _ = (1, 2)    # only x is bound; 2 is discarded
print(x)        # 1
```

Mutable Destructuring

Omit `@const` to declare mutable variables.

```
a, b = (10, 20)
a = 99
print(a)    # 99
```

Rules

- The number of variables on the left must match the number of elements in the tuple.
 - Each variable follows the same `@const`/mutable rules as a regular declaration.
 - Nested tuple destructuring is not supported.
-

[<- 01 - Getting Started](#) / [Next -> 03 - Operators](#)

[English](#) | [日本語](#) | [繁體中文](#)

03 - Operators

[<- 02 - Variables and Types](#) / Next -> [04 - Control Flow](#)

Arithmetic Operators

Operator	Description	Example	Result
+	Addition	3 + 2	5
-	Subtraction	3 - 2	1
*	Multiplication / string repetition	3 * 2	6
/	Division (always float)	7 / 2	3.5
//	Floor division (int if both int, float if either is float)	7 // 2	3
%	Modulo	7 % 3	1
**	Exponentiation (always float)	2 ** 10	1024

```
a = 10
b = 3
```

```
print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.3333... (float)
print(a // b)   # 3 (int)
print(a % b)    # 1
print(2 ** 8)   # 256 (float)
```

Overflow safety: Arithmetic on `int` (+, -, *, unary -) raises a runtime error if the result overflows the 64-bit signed range. Constant expressions that overflow are caught at compile time. Low-level types (`i32`, `u8`, etc.) wrap silently — use `checked_add`/`saturating_add`/`wrapping_add` for explicit overflow control.

Comparison Operators

All comparison operators return a `bool` value.

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
<	Less than	a < b
<=	Less than or equal to	a <= b
>	Greater than	a > b
>=	Greater than or equal to	a >= b

```
x = 5
y = 10
```

```
print(x == y)   # false
print(x != y)   # true
print(x < y)     # true
```

```
print(x <= y)    # true
print(x > y)     # false
print(x >= y)    # false
```

Comparison operators also work on strings (lexicographic comparison).

```
print("abc" == "abc") # true
print("abc" < "abd")  # true
print("b" > "a")      # true
```

Logical Operators

Operator	Description	Example
and	Logical AND	a and b
or	Logical OR	a or b
not	Logical NOT	not a

```
t = true
f = false
```

```
print(t and f) # false
print(t or f)  # true
print(not t)   # false
print(not f)   # true
```

Bitwise Operators

Bitwise operators can only be used with the `int` type.

Operator	Description	Example
&	Bitwise AND	5 & 3 -> 1
\	Bitwise OR	5 \ 3 -> 7
^	Bitwise XOR	5 ^ 3 -> 6
~	Bitwise NOT (unary)	~5 -> -6
<<	Left shift	1 << 3 -> 8
>>	Right shift (arithmetic)	8 >> 2 -> 2
>>>	Logical right shift	-1 >>> 1 -> 9223372036854775807

```
a = 0b1010 # 10
b = 0b1100 # 12
```

```
print(a & b)    # 8 (0b1000)
print(a | b)    # 14 (0b1110)
print(a ^ b)    # 6 (0b0110)
print(~a)       # -11
print(1 << 4)   # 16
print(32 >> 2)  # 8
```

Compound Assignment Operators

Shorthand notation for updating the value of a variable.

Operator	Description	Equivalent Expression
<code>+=</code>	Addition assignment	<code>x = x + n</code>
<code>-=</code>	Subtraction assignment	<code>x = x - n</code>
<code>*=</code>	Multiplication assignment	<code>x = x * n</code>
<code>/=</code>	Division assignment	<code>x = x / n</code>
<code>%=</code>	Modulo assignment	<code>x = x % n</code>
<code>//=</code>	Floor division assignment	<code>x = x // n</code>
<code>**=</code>	Exponentiation assignment	<code>x = x ** n</code>
<code>&=</code>	Bitwise AND assignment	<code>x = x & n</code>
<code> =</code>	Bitwise OR assignment	<code>x = x n</code>
<code>^=</code>	Bitwise XOR assignment	<code>x = x ^ n</code>
<code><<=</code>	Left shift assignment	<code>x = x << n</code>
<code>>>=</code>	Right shift assignment	<code>x = x >> n</code>

```
x = 10
x += 5    # x == 15
x -= 3    # x == 12
x *= 2    # x == 24
x /= 4    # x == 6 (becomes float)
```

Increment / Decrement Operators

Shorthand for incrementing or decrementing a variable by 1.

Operator	Description	Equivalent Expression
<code>x++</code>	Increment by 1	<code>x = x + 1</code>
<code>x--</code>	Decrement by 1	<code>x = x - 1</code>

```
count = 0
count++    # count == 1
count++    # count == 2
count--    # count == 1
```

Note: These are statement-only operators. They cannot be used inside expressions.

Type Promotion Rules

The following describes the behavior when `int` and `float` are mixed in operations.

```
# + - * produce float if either operand is float
print(1 + 2)      # 3 (int)
print(1 + 2.0)    # 3 (float)
print(1.0 + 2)    # 3 (float)

# / always produces float
print(4 / 2)      # 2 (float)
```

```

# // produces int if both int, float if either is float
print(7 // 2)      # 3 (int)
print(7.0 // 2)    # 3 (float)

# ** always produces float
print(2 ** 3)      # 8 (float)

# % produces int if both operands are int, float if either is float
print(7 % 3)       # 1 (int)
print(7.5 % 2)     # 1.5 (float)

# + concatenates strings if both operands are str
print("foo" + "bar") # "foobar"

# * repeats a string if one operand is str and the other is int
print("ab" * 3)     # "ababab"
print(3 * "ab")     # "ababab"

```

Membership Operators

Operator	Description	Example
in	Membership test	2 in {1, 2, 3} -> true
not in	Negated membership test	4 not in {1, 2, 3} -> true

```

s = {1, 2, 3}
print(2 in s)      # true
print(4 not in s)  # true

```

[<- 02 - Variables and Types](#) / [Next -> 04 - Control Flow](#)

[English](#) | [日本語](#) | [繁體中文](#)

Control Flow

[<- Prev: Operators](#) | [Next: Functions ->](#)

if / else

Use if to branch execution based on conditions.

```
x = 10
```

```

if x > 0:
    print(x)
else:
    print(0)

```

- else is optional.
- Conditions are not limited to bool values. For int, 0 is treated as false and non-0 as true.
- if statements can be nested.

```

a = 5
b = 3

if a > 0:
    if b > 0:
        print(a + b)    # 8

```

case

Use `case:` for multi-branch conditionals without a subject, or `case value:` for pattern matching with a subject. Both forms replace the old `when` and `match` keywords.

Conditional case:

```

x = -2

case:
    x > 0:
        print("positive")
    x < 0:
        print("negative")
    -:
        print("zero")

```

This is the preferred form when you would otherwise chain multiple branches.

Pattern case value:

```

enum Color:
    Red
    Green
    Blue

c = Color::Green
case c:
    Color::Red:
        print("red")
    Color::Green:
        print("green")
    Color::Blue:
        print("blue")

```

Use this form to destructure enums, `Option`, `Result`, and literal patterns safely.

case: Expressions

`case:` can also be used as an expression. The `_ =>` wildcard arm is required.

```

label = case:
    score >= 90 => "A"
    score >= 80 => "B"
    _ => "C"

```

This replaces nested ternary expressions and keeps multi-branch value selection readable.

case value: Expressions

`case value:` can also be used as an expression with `=>` to return a value from each arm.

```
res = case x:
    Some(v) => v
    None    => 0

label = case direction:
    Direction::North => "N"
    Direction::South => "S"
    Direction::East  => "E"
    Direction::West  => "W"
```

```
category = case score:
    n if n >= 90 => "A"
    n if n >= 80 => "B"
    _            => "F"
```

Case expressions support all the same patterns as case statements: literals, variables, enums, `Option`, `Result`, OR patterns (`|`), guards (`if`), and wildcards (`_`). The case must be exhaustive.

if Expressions

For simple two-branch conditionals that produce a value, use the `if` expression:

```
abs_val = if x >= 0 => x else -x
label = if score >= 90 => "A" else "B"
```

The `else` branch is required and both branches must produce the same type. For multi-branch expressions, use `case:` instead.

while Loop

Repeatedly executes a block as long as the condition is true.

```
i = 3
while i > 0:
    print(i)
    i = i - 1
# 3
# 2
# 1
```

for Loop and range

You can iterate over a list or using `range`.

```
for x in [1, 2, 3]:
    print(x)
# 1
# 2
# 3
```

`range(n)` generates integers from 0 to `n - 1`.

```

for i in range(5):
    print(i)
# 0
# 1
# 2
# 3
# 4

```

`range(start, end)` generates integers from `start` to `end - 1`.

```

for i in range(2, 5):
    print(i)
# 2
# 3
# 4

```

The `..` range operator creates an inclusive range: `1 .. 3` produces `[1, 2, 3]`.

```

for i in 1 .. 3:
    print(i)
# 1
# 2
# 3

```

You can iterate over map key-value pairs with `for k, v in map:`

```

m = {"x": 10, "y": 20}
for k, v in m:
    print(k)
    print(v)

```

break and continue

`break` immediately exits the loop. `continue` skips the current iteration and proceeds to the next one.

```

for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
# 1
# 3

```

They can also be used with `while` loops.

```

n = 0
while true:
    n = n + 1
    if n % 2 == 0:
        continue
    if n > 7:
        break
    print(n)
# 1
# 3

```

```
# 5
# 7
```

Note: In nested loops, `break` / `continue` only affect the innermost loop. Using them outside a loop results in a compile error.

Nesting Example

`for` and `while` loops can be nested.

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i * 10 + j)
```

```
# 11
# 13
# 21
# 23
# 31
# 33
```

Scope Rules

Control flow blocks have their own scope.

Block Scope

Variables declared inside a block cannot be referenced from outside the block.

```
if true:
    inner = 42
# Referencing inner here causes a compile error
```

Referencing and Reassigning Outer Variables

You can reference and reassign outer variables from within a block.

```
count = 0
for i in range(5):
    count = count + i
print(count)    # 10
```

Inner Scope Reassignment

Assigning to a variable inside a block modifies the outer variable (Python-style scoping). There is no shadowing — the inner assignment changes the same variable.

```
x = 1
if true:
    x = 99
    print(x)    # 99
print(x)        # 99
```

Pattern Matching

`case value:` safely branches on enums, `Option`, `Result`, and literals.

```
enum Color:
    Red
    Green
    Blue

c = Color::Green
case c:
    Color::Red:
        print("red")
    Color::Green:
        print("green")
    Color::Blue:
        print("blue")
# green
```

Option Matching

Use `case value:` to safely handle both the `Some` and `None` cases.

```
x: Option<int> = Some(42)
case x:
    Some(v):
        print(v)
    None:
        print("nothing")
# 42
```

Wildcards and Literals

`_` is a wildcard pattern that matches anything. You can also match against literal values (numbers, strings, booleans).

```
n = 5
case n:
    0:
        print("zero")
    1:
        print("one")
    _:
        print("other")
# other
```

Guard Clauses

You can add guard conditions with `if`.

```
case n:
    x if x > 0:
        print("positive")
    x if x < 0:
        print("negative")
    _:
        print("zero")
```

Note: `case value:` must be exhaustive. For enums, all variants must be covered. For Options, both `Some` and `None` are required. For literals, a `_` wildcard is needed.

[<- Prev: Operators](#) | [Next: Functions ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Functions

[<- Prev: Control Flow](#) | [Next: Records and Enums ->](#)

Basic Function Definition

Functions are defined with the `function` keyword. Parameter types use the `name: type` format. If a type is omitted, it defaults to `any`. The return type is specified after `->`.

```
function add(a: int, b: int) -> int:
  return a + b
```

- Parameter type declarations are recommended. If omitted, the type defaults to `any`.
 - The return type is specified after `->`.
 - Use `return` to return a value.
-

Calling Functions

Call a defined function by its name with arguments.

```
function multiply(x: int, y: int) -> int:
  return x * y
```

```
result = multiply(3, 4)
print(result)    # 12
```

Recursive Functions

Functions can call themselves (recursion).

```
function factorial(n: int) -> int:
  if n <= 1:
    return 1
  return n * factorial(n - 1)
```

```
print(factorial(5))    # 120
print(factorial(0))    # 1
```

Function Overloading

You can define multiple functions with the same name but different parameter counts or types.

```
function add(a: int, b: int) -> int:
    return a + b

function add(a: float, b: float) -> float:
    return a + b

print(add(1, 2))           # 3
print(add(1.5, 2.5))      # 4
```

The appropriate function is automatically selected based on the argument types at the call site.

Note: Defining functions with identical parameter types but different return types causes a compile error.

Omitting the Return Type (Unit Type)

If a function does not need to return a value, you can omit `->`. In this case, the function returns the Unit type.

```
function greet():
    print(42)

greet()    # 42
```

This is the simplest form of a function with no parameters and no return value.

Default Arguments

Parameters can have default values. When a caller omits those arguments, the default is used.

```
function greet(name: str, greeting: str = "Hello") -> str:
    return f"{greeting}, {name}"

print(greet("Alice"))           # Hello, Alice
print(greet("Bob", "Good morning")) # Good morning, Bob
```

You can have multiple default parameters:

```
function connect(host: str, port: int = 8080, timeout: int = 30) -> str:
    return f"{host}:{port} (timeout={timeout})"

print(connect("localhost"))           # localhost:8080 (timeout=30)
print(connect("localhost", 3000))     # localhost:3000 (timeout=30)
print(connect("localhost", 3000, 10)) # localhost:3000 (timeout=10)
```

Why default arguments? They let you keep a simple call site for common cases while allowing customization when needed — no need for multiple overloads.

Note: Parameters with defaults must come after parameters without defaults.

Lambda Functions

Lambda functions let you write functions as expressions. Single-expression lambdas use the form `(parameters) => expression`, while block lambdas use `(parameters):` followed by an indented block. In both cases, the return type is automatically inferred.

Single-Expression Lambda

```
double = (x: int) => x * 2
print(double(5)) # 10
```

```
add = (a: int, b: int) => a + b
print(add(3, 4)) # 7
```

No-Parameter Lambda

```
answer = () => 42
print(answer()) # 42
```

Multi-Line Lambda

You can write multiple statements by adding a newline after `:` and indenting.

```
abs = (x: int):
    if x < 0:
        return -x
    return x

print(abs(-5)) # 5
print(abs(3))  # 3
```

Why lambdas? They are ideal for short, throwaway functions —especially as arguments to higher-order functions like `filter` and `map` (see below).

Closures

Lambda functions can capture variables from the scope in which they are defined. This combination of a function and its captured environment is called a **closure**.

```
offset = 10
add_offset = (x: int) => x + offset
print(add_offset(5)) # 15
```

Closures capture variables **by value** — changing the original variable after creating the closure does not affect the closure's copy.

```
base = 10
f = (x: int) => x + base
base = 999
print(f(1)) # 11 (still uses the captured value 10)
```

This works in both directions — mutations inside the closure do not affect the outer variable either:

```
counter = 0
items = [1, 2, 3]
items.map((x: int):
    counter += x # Only modifies the closure's local copy
    return x
)
print(counter) # 0 (outer variable unchanged)
```

Why capture by value? It ensures safety and predictability —you can always reason about a variable's value by looking at the current scope alone, without worrying about mutations

happening inside closures. **Why closures?** They let you create specialized functions on the fly. For example, you can create a family of adder functions from a single template.

Higher-Order Functions

You can define functions that take other functions as arguments. Function types are written as `function(parameter_types) -> return_type`.

```
function apply(f: function(int) -> int, x: int) -> int:
    return f(x)
```

```
double = (x: int) => x * 2
print(apply(double, 3))           # 6
print(apply((n: int) => n + 1, 10)) # 11
```

Functions as Values

Named functions can also be bound to variables or passed as arguments —they behave just like lambdas.

```
function square(x: int) -> int:
    return x * x

# Pass a named function as an argument
print(apply(square, 4)) # 16

# Bind to a variable
sq = square
print(sq(5)) # 25
```

Why higher-order functions? They let you separate **what** to do from **how** to do it. The same `apply` function works with any transformation, making code more reusable. You have already seen this pattern in [Collections](#) with `filter`, `map`, and `reduce`.

UFCS (Uniform Function Call Syntax)

With UFCS, you can write `f(a, b)` as `a.f(b)`. The first argument is moved before the dot, enabling a method-chaining style.

```
function add(a: int, b: int) -> int:
    return a + b
```

```
x = 1
print(x.add(2)) # add(x, 2) -> 3
```

Chained Calls

UFCS really shines when you chain multiple calls — it reads left-to-right instead of inside-out:

```
function double(n: int) -> int:
    return n * 2

# Chained (reads naturally: "take x, add 2, then double")
print(x.add(2).double()) # 6
```

```
# Equivalent nested calls (harder to read)
print(double(add(x, 2))) # 6
```

Why UFCS? It turns deeply nested function calls into readable left-to-right pipelines. You have already seen this with iterator chains like `xs.iter().filter(...).map(...).to_list()`.

Exercises

1. **Default arguments:** Write a function `format_price(amount: int, currency: str = "USD", decimals: int = 2) -> str` that formats a price. Verify that `format_price(42)` and `format_price(42, "EUR")` both work.
 2. **Higher-order function:** Write a function `apply_twice(f: function(int) -> int, x: int) -> int` that applies `f` to `x` twice (i.e., `f(f(x))`). Test it with `(x: int) => x + 1` and verify that `apply_twice((x: int) => x + 1, 5)` returns 7.
 3. **UFCS chaining:** Define `inc(n: int) -> int` (adds 1) and `triple(n: int) -> int` (multiplies by 3). Use UFCS to write `5.inc().triple()` and verify the result is 18.
-

[<- Prev: Control Flow](#) | [Next: Records and Enums ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Records and Enums

[<- Prev: Functions](#) | [Next: Collections and Iterators ->](#)

Defining Records

Records are defined with the `record` keyword. Each field is described in the `name: type` format.

```
record Point:
  x: int
  y: int
```

Records are value types allocated on the stack.

Creating Instances

Create an instance by calling the record name as a function. Arguments are specified in the order the fields are defined.

```
p = Point(10, 20)
```

Field Access (Dot Notation)

Fields are accessed using dot notation.

```
p = Point(10, 20)
print(p.x) # 10
print(p.y) # 20
```

Records can be printed directly:

```
p = Point(10, 20)
print(p)    # Point(x: 10, y: 20)
```

Field Assignment

Fields of mutable variables (declared without `@const`) can be reassigned.

```
p = Point(10, 20)
p.x = 100
print(p.x)    # 100
```

Note: Assigning to fields of a `@const` variable causes a compile error.

Records as Function Parameters

Records can be passed as function arguments.

```
record Point:
  x: int
  y: int

function distance_x(a: Point, b: Point) -> int:
  return a.x - b.x

p1 = Point(10, 3)
p2 = Point(4, 7)
print(distance_x(p1, p2))    # 6
```

Nested Records

A record's field can be another record.

```
record Point:
  x: int
  y: int

record Line:
  start: Point
  end: Point

line = Line(Point(0, 0), Point(10, 5))
print(line.start.x)    # 0
print(line.end.x)      # 10
```

You can access nested fields by chaining dot notation.

Enums

Enums are defined with the `enum` keyword. Each variant is treated as a named constant.

Definition

```
enum Color:
    Red
    Green
    Blue
```

Usage

Variants are accessed with `::`.

```
c = Color::Red
print(c)    # Red
```

Comparison

Variants can be compared using `==` and `!=`.

```
case:
    c == Color::Red:
        print("red!")
    c == Color::Green:
        print("green!")
    -:
        print("blue!")
```

Function Parameters

Enum names can be used as function parameter types.

```
function describe(c: Color) -> str:
    if c == Color::Red:
        return "warm"
    return "cool"
```

Enum with Associated Data (ADT)

Enum variants can carry associated values. This lets a single enum represent a family of different shapes of data — a pattern known as an **Algebraic Data Type (ADT)**.

```
enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point
```

Named fields are documentation-only — they make definitions self-describing. Unnamed syntax (`Circle(float)`) is also valid.

Constructing ADT Variants

Construction is always positional, regardless of whether fields are named.

```
c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point
```

Matching ADT Variants

Use `case` to extract the associated data from ADT variants. Bindings use your chosen variable names, not the field names. This connects directly with the pattern matching you learned in [Control Flow](#).

```
function describe(s: Shape) -> str:
  case s:
    Shape::Circle(r):
      return f"circle with radius {r}"
    Shape::Rectangle(w, h):
      return f"rectangle {w}x{h}"
    Shape::Point:
      return "point"

print(describe(Shape::Circle(3.14)))      # circle with radius 3.14
print(describe(Shape::Rectangle(4.0, 5.0))) # rectangle 4.0x5.0
```

Why ADTs? They let you model data that can be “one of several shapes” in a type-safe way. The compiler ensures you handle every variant when pattern matching, catching missing cases at compile time.

Generic Enums

Enums can take type parameters, making them reusable across different payload types.

```
enum MyOption<T>:
  MySome(T)
  MyNone
```

Usage

```
a = MyOption<int>::MySome(42)
b: MyOption<int> = MyOption<int>::MyNone
```

```
case a:
  MyOption::MySome(v):
    print(v)      # 42
  MyOption::MyNone:
    print("none")
```

Note: Ry’ s built-in `Option<T>` and `Result<T, E>` types work exactly like this. You will learn about them in [Error Handling](#).

Operator Overloading

You can define operators for custom types using the `function operator` syntax. This lets your records work naturally with `+`, `==`, and other operators.

Binary Operators

Takes two parameters.

```
record Vec2:
  x: int
  y: int
```

```

function operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

v1 = Vec2(1, 2)
v2 = Vec2(3, 4)
v3 = v1 + v2
print(v3.x)      # 4
print(v1 == v2)  # false

```

Unary Operators

Takes one parameter.

```

function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)

```

Supported Operators

Category	Operators
Arithmetic	+, -, *, /, %, **, //
Comparison	==, !=, <, <=, >, >=
Bitwise	&, \ , ^, ~, <<, >>, >>>
Logical	and, or, not
Membership	in
Index access	[]
Index assignment	[] =
Function call	()
Type cast	as
Compound assignment	+=, -=, *=, /=, %=, //, **, &=, \ =, ^=, <<=, >>=

Why operator overloading? It gives domain types natural syntax. A `Vec2 + Vec2` reads better than `vec2_add(a, b)`, and `==` lets your types work seamlessly with `case` and comparisons.

Exercises

1. **ADT:** Define an `Animal` enum with variants `Dog(name: str)`, `Cat(name: str, indoor: bool)`, and `Fish`. Write a `describe(a: Animal) -> str` function that uses `case` to return a description for each variant.
2. **Operator overloading:** Define a `Money` record with `amount: int` and `currency: str`. Overload `+` so that adding two `Money` values with the same currency returns a new `Money` with the summed amount.

[<- Prev: Functions](#) | [Next: Collections and Iterators ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Collections and Iterators

[<- Prev: Records and Enums](#) | [Next: Error Handling ->](#)

Ry has four collection types: **Tuples**, **Lists**, **Maps**, and **Sets**.

Tuples

A tuple is an immutable data structure that groups multiple values together. It can hold elements of different types.

Creation

```
t = (1, 3.14)
```

Type Annotation

```
t: (int, float) = (1, 3.14)
```

Element Access

Elements are accessed by index using `.0`, `.1`, etc.

```
t = (1, 3.14)
print(t.0)    # 1
print(t.1)    # 3.14
```

As Function Return Values

Tuples are useful when you want to return multiple values.

```
function swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0) # 2
print(result.1) # 1
```

Limitations

- Accessing an out-of-bounds index (e.g., `.2` on a tuple with 2 elements) causes a compile error.
 - Passing a tuple directly to `print` causes an error. Pass each element individually.
-

Lists

A list is a variable-length data structure containing elements of the same type.

Creation

```
xs = [1, 2, 3]
```

Type Annotation

```
xs: List<int> = [1, 2, 3]
```

Index Access

```
print(xs[0])    # 1

i = 1
print(xs[i])    # 2
```

Index Assignment

```
xs[0] = 99
```

length

```
print(length(xs))    # 3
```

print

```
print(xs)    # [1, 2, 3]
```

Iteration with for

```
for x in xs:
    print(x)
```

Function Parameters

```
function first(xs: List<int>) -> int:
    return xs[0]
```

filter, map, sort

Lists support filter, map, and sort operations. These return new lists without modifying the original.

```
xs = [1, 2, 3, 4, 5]
```

```
# filter: keep elements matching a condition
evens = filter(xs, (x: int) => x > 3)
print(evens)    # [4, 5]
```

```
# map: transform each element
doubled = map(xs, (x: int) => x * 2)
print(doubled)    # [2, 4, 6, 8, 10]
```

```
# sort: sort in ascending order (default)
sorted = sort([3, 1, 2])
print(sorted)    # [1, 2, 3]
```

```
# Chaining with UFCS (Uniform Function Call Syntax)
# x.f(args) is equivalent to f(x, args)
result = xs.filter((x: int) => x > 1).map((x: int) => x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce, fold

reduce accumulates a list into a single value starting from the first element. fold does the same but with an explicit initial value.

```

xs = [1, 2, 3, 4, 5]

# reduce: start from first element
total = reduce(xs, (a: int, b: int) => a + b)
print(total)    # 15

# fold: provide an explicit initial value
total2 = fold(xs, 0, (a: int, b: int) => a + b)
print(total2)   # 15

```

any, all

`any` returns `true` if at least one element satisfies the predicate. `all` returns `true` if every element does.

```

xs = [1, 2, 3, 4, 5]

print(any(xs, (x: int) => x > 4))    # true
print(any(xs, (x: int) => x > 9))    # false

print(all(xs, (x: int) => x > 0))    # true
print(all(xs, (x: int) => x > 3))    # false

```

sum, min, max

```

xs = [3, 1, 4, 1, 5]
print(sum(xs))    # 14
print(min(xs))    # 1
print(max(xs))    # 5

```

first, last, is_empty

```

xs = [10, 20, 30]
print(first(xs))    # Some(10)
print(last(xs))     # Some(30)
print(is_empty(xs)) # false

```

enumerate, zip

`enumerate` pairs each element with its index. `zip` combines two lists element-by-element.

```

xs = [10, 20, 30]
indexed = enumerate(xs)
# [(0, 10), (1, 20), (2, 30)]
for p in indexed:
    print(p.0)
    print(p.1)

ys = ["a", "b", "c"]
zipped = zip(xs, ys)
# [(10, "a"), (20, "b"), (30, "c")]

```

Limitations

- All elements must be of the same type. Mixing different types is not allowed.
- An empty list `[]` causes an error.
- Out-of-bounds access results in a runtime error (`exit(1)`).
- Supported element types are `int`, `float`, `bool`, and `str`.

Maps

A map is an associative array that manages key-value pairs.

Creation

```
m = {"a": 1, "b": 2}
```

Type Annotation

```
m: Map<str, int> = {"a": 1, "b": 2}
```

Key Access

```
print(m["a"])    # 1
```

Insertion / Update

Assigning to a new key inserts it; assigning to an existing key updates it.

```
m["c"] = 3        # Insert new entry
m["a"] = 99       # Update existing entry
```

length

```
print(length(m))  # 3
```

print

```
print(m)          # {a: 99, b: 2, c: 3}
```

has__key

Checks whether a key exists.

```
print(has_key(m, "a"))  # true
```

keys, values

`keys` returns a list of all keys. `values` returns a list of all values.

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))      # ["a", "b", "c"]
print(values(m))    # [1, 2, 3]
```

Function Parameters

```
function get_val(m: Map<str, int>, k: str) -> int:
    return m[k]
```

Limitations

- All keys must be of the same type, and all values must be of the same type.
- An empty map requires a type annotation.
- Accessing a nonexistent key results in a runtime error (`exit(1)`).

Sets

A set is a collection that holds elements of the same type without duplicates.

Creation

```
s = {1, 2, 3}
```

Type Annotation

```
s: Set<int> = {1, 2, 3}
```

in Operator

Use the `in` operator to check if an element is in the set.

```
print(2 in s)    # true
print(5 in s)    # false
```

add / remove

```
add(s, 4)        # Add element
remove(s, 1)     # Remove element
add(s, 2)        # Ignored since it already exists
```

length / print

```
print(length(s)) # 3
print(s)         # {2, 3, 4}
```

Iteration with for

```
for x in s:
    print(x)
```

Empty Set

An empty set requires a type annotation.

```
empty: Set<int> = {}
```

Limitations

- All elements must be of the same type.
 - Supported element types are `int`, `float`, `bool`, and `str`.
-

Iterators

Iterators provide a **lazy** way to process collections. Instead of creating intermediate lists at each step, iterators process elements one at a time through a pipeline.

Why lazy iterators? When you chain `filter` and `map` on a list directly, each step creates a new intermediate list. With iterators, elements flow through the entire pipeline one at a time — no intermediate allocations. This matters when processing large collections or when you only need the first few results (using `take`).

Creating and Consuming

Call `iter()` on a collection to get an iterator, and `to_list()` to materialize the results back into a list:

```
xs = [1, 2, 3]
ys = to_list(iter(xs))    # [1, 2, 3]
```

Chaining Operations

You can chain `filter`, `map`, and `take` to build pipelines. This uses the UFCS chaining style you learned in [Functions](#):

```
result = to_list(take(map(filter(iter([1, 2, 3, 4, 5]), (x: int) => x > 2), (x: int) => x * 2), 2))
print(result)    # [6, 8]
```

UFCS chaining style (equivalent):

```
result = [1, 2, 3, 4, 5]
    .iter()
    .filter((x: int) => x > 2)
    .map((x: int) => x * 2)
    .take(2)
    .to_list()
print(result)    # [6, 8]
```

Here is a more realistic example — processing a list of scores:

```
scores = [85, 42, 93, 67, 78, 55, 91]

# Get the top 3 passing scores (>= 60), doubled for bonus
top_bonus = to_list(take(map(filter(iter(scores), (s: int) => s >= 60), (s: int) => s * 2), 3))
print(top_bonus)    # [170, 186, 134]
```

Manual Iteration with next()

`next()` returns an `Option` — `Some(value)` if there is a next element, or `None` when the iterator is exhausted. You will learn more about `Option` in [Error Handling](#).

```
it = iter([10, 20])
print(next(it))    # Some(10)
print(next(it))    # Some(20)
print(next(it))    # None
```

For Loops

Iterators work directly in `for` loops:

```
for x in filter(iter([1, 2, 3]), (x: int) => x > 1):
    print(x)        # 2, 3
```

Iterating over Maps and Sets

Maps produce key-value tuples. Sets produce individual elements:

```
for k, v in iter({"a": 1, "b": 2}):
    print(f"{k} = {v}")

for x in iter({10, 20, 30}):
    print(x)
```

Common Mistakes

- **Forgetting `to_list()`:** An iterator pipeline by itself does nothing — it is lazy. You must consume it with `to_list()`, a `for` loop, or `next()`.
 - **Calling `to_list()` too early:** Placing `to_list()` before `filter()` defeats the purpose of lazy evaluation since it materializes all elements first.
-

Exercises

1. **Iterator pipeline:** Given `xs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, use an iterator pipeline to compute the sum of even numbers. (Hint: use `.filter()` then `.to_list()` and `sum()`.)
 2. **Manual iteration:** Create an iterator over `[100, 200, 300]` and use `next()` in a `case` block to handle the `Some` and `None` cases.
-

[<- Prev: Records and Enums](#) | [Next: Error Handling ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Error Handling

[<- Prev: Collections and Iterators](#) | [Next: Packages ->](#)

Ry provides three complementary strategies for dealing with errors and absent values: **Option** (a value may be missing), **Result** (an operation may fail), and **Design by Contract** (prevent invalid states at the boundary). This tutorial covers all three and when to use each.

Option Type

`Option<T>` represents a value that may or may not exist. It has two variants: `Some(value)` and `None`.

```
x: Option<int> = Some(42)
print(x)      # Some(42)
```

```
y: Option<int> = None
print(y)      # None
```

Extracting the Value

Use `case` to safely extract the inner value and handle the `None` case. This uses the pattern matching you learned in [Control Flow](#):

```
case x:
  Some(v):
    print(v)      # 42
  None:
    print("nothing")
```

Why Option? It makes the possibility of absence explicit in the type system. Instead of returning a “magic value” like `-1` or checking for `null`, the caller must handle the `None` case — the compiler ensures it.

Where You Encounter Option

You have already seen `Option` in action: `iterator.next()` returns `Option<T>`, giving `Some(element)` for each element and `None` when the iterator is exhausted.

Result Type

`Result<T, E>` is used for operations that may fail. Return `Ok(value)` for success and `Err(error)` for failure.

```
function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)
```

Handling Results with case

```
r = divide(10, 0)
case r:
    Ok(v):
        print(v)
    Err(e):
        print(e.message)    # division by zero
```

The ? Operator (Error Propagation)

When calling a function that returns `Result` from another function that also returns `Result`, you can use `?` to propagate errors automatically. If the value is `Ok`, it is unwrapped; if `Err`, the function returns immediately with that error.

```
function safe_divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

function divide_and_add(a: int, b: int) -> Result<int, Error>:
    v = safe_divide(a, b)?    # Returns Err early if b == 0
    return Ok(v + 1)
```

This is equivalent to writing:

```
function divide_and_add(a: int, b: int) -> Result<int, Error>:
    case safe_divide(a, b):
        Ok(v):
            return Ok(v + 1)
        Err(e):
            return Err(e)
```

The `?` operator removes the boilerplate, letting you chain multiple fallible operations cleanly:

```
function compute(a: int, b: int, c: int) -> Result<int, Error>:
    x = safe_divide(a, b)?
    y = safe_divide(x, c)?
    return Ok(y + 1)
```

Method Chaining with `and_then` and `map`

When you need to chain several `Result`-returning operations but cannot use `?` (for example, outside a function that returns `Result`), you can use `and_then` and `map` to avoid deeply nested `case` statements.

`and_then` — chains operations that themselves return `Result`:

```
# Instead of nesting case 3 levels deep:
result = safe_divide(100, 2)
    .and_then((v: int) => safe_divide(v, 5))
    .and_then((v: int) => safe_divide(v, 2))

case result:
  Ok(v):  print(v)      # 5
  Err(e): print(e.message)
```

`map` — transforms the `Ok` value without changing the `Result` wrapper:

```
result = safe_divide(10, 2)
    .map((v: int) => v * 10)

case result:
  Ok(v):  print(v)      # 50
  Err(e): print(e.message)
```

Both methods short-circuit on `Err` — if any step in the chain fails, the error propagates through without executing the remaining closures.

You can mix `and_then` and `map` in a single chain:

```
result = safe_divide(100, 10)
    .and_then((v: int) => safe_divide(v, 2))
    .map((v: int) => v + 100)
# Ok(105)
```

Why `Result`? It makes error handling explicit without exceptions. The type signature tells you exactly which functions can fail, and the `?` operator keeps the code concise.

Common mistake: Using `?` in a function that does not return `Result` causes a compile error. The `?` operator can only be used in functions whose return type is `Result`.

Design by Contract

Ry supports Eiffel-style Design by Contract with preconditions (`require`), postconditions (`ensure`), and record invariants (`invariant`). While `Option` and `Result` handle errors at runtime, contracts **prevent** invalid states from occurring in the first place.

For the full specification, see [Design by Contract Reference](#).

Preconditions (`require`)

Use `require` to specify conditions that must be true when a function is called:

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  new_balance: int = balance + amount
  return new_balance
```

If any precondition fails, the program terminates with:

Contract violation: require failed in deposit()

Postconditions (ensure)

Use `ensure` to specify conditions that must be true when a function returns. The return value is bound to a user-chosen variable name:

```
function abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

Since function arguments are immutable in Ry, you can reference them directly in `ensure` blocks:

```
function increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

For tuple returns, use multiple variable names:

```
function divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

Combining require and ensure

Both can be used together. `require` must come before `ensure`:

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

Record Invariants (invariant)

Use `invariant` to specify conditions that must always hold for a record. Invariants are checked after construction and after every field assignment:

```
record BankAccount:
  balance: int
  min_balance: int
  invariant:
    balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
# a.balance = -1          # Contract violation: invariant failed
```

Why contracts? They document and enforce assumptions directly in code. If a function requires `amount > 0`, the contract catches violations immediately at the call site — not deep inside the function body where the symptom appears.

Contract Rules

- **require** and **ensure** blocks are optional and appear before the function body.
- **require** must come before **ensure** when both are present.
- **ensure** requires a variable binding (e.g., **ensure** `v:`) to name the return value.
- **invariant** appears at the end of a **record** definition, after all field declarations.
- All contract violations terminate with `exit(1)`.

When to Use What

Strategy	Use When	Example
Option	A value may legitimately be absent	Looking up a key, <code>iterator.next()</code>
Result	An operation can fail with a meaningful error	File I/O, parsing, network calls
Contract	Invalid input should never happen (programmer error)	Negative deposit, out-of-bounds index

Rules of thumb: - Use **Result** for operations that can fail due to external factors (user input, file system, network). - Use **Option** when “nothing” is a normal, expected outcome. - Use **contracts** to catch programmer mistakes early — they are assertions, not error handling.

Common Mistakes

1. **Ignoring a Result:** If you call a function returning **Result** and don't handle it, you lose the error information.
2. **Using ? in a non-Result function:** The `?` operator requires the enclosing function to return **Result**.
3. **Confusing Option and Result:** **Option** has `Some/None`; **Result** has `Ok/Err`. They serve different purposes.

Exercises

1. **Result with ?:** Write a function `parse_and_double(s: str) -> Result<int, Error>` that parses a string to an integer using a helper function and doubles it. Use `?` for error propagation.
2. **Contracts:** Write a function `withdraw(amount: int, balance: int) -> int` with **require** that `amount > 0` and `amount <= balance`, and **ensure** that the result is non-negative.
3. **Option handling:** Write a function that takes a `List<int>` and returns the first even number as `Option<int>`, returning `None` if no even numbers exist.

[<- Prev: Collections and Iterators](#) | [Next: Packages ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Packages

[<- Prev: Error Handling](#) | [Next: Concurrency ->](#)

Ry uses a package system to organize code across files and directories. For the full specification, see [Package Reference](#).

from/import Syntax

Use the `from` syntax to import functions from another file.

```
from math import sqrt, PI    # Selective import
from math                    # Full import (all definitions)
```

This makes functions defined in `math.ry` available for use.

Subdirectories (Dot-Separated Paths)

Use dot-separated paths to specify packages in subdirectories.

```
from utils.calc import add    # Import from utils/calc.ry
```

Each dot corresponds to a directory separator.

Directory Packages

A package can be either a single `.ry` file or a directory containing multiple `.ry` files. When a package resolves to a directory, all `.ry` files in it are automatically loaded.

```
mypackage/
  calc.ry      # function add(), function sub()
  string.ry    # function concat()

from mypackage          # imports add, sub, concat
from mypackage import add # imports only add
```

No special entry file (like `__init__.py`) is needed. Files starting with `_` are excluded.

Relative Imports

Use a leading `.` to import relative to the current file's directory. This is especially useful for test files that need to import from sibling modules.

```
from .helper import greet    # Import from helper.ry in the same directory
from .utils import add       # Import from utils/ subdirectory
from .utils.calc import mul   # Import from utils/calc/ nested subdirectory
from . import add, sub        # Import symbols from the current directory package
```

Relative imports resolve **only** against the current file's directory — the standard library and other search paths are not searched. This prevents name collisions when your project has modules with the same name as standard library packages.

```
# If your project has src/math/stats.ry:
from .math import mean    # Always resolves to your local math package
from math import sqrt     # Resolves to the standard library math package
```

Note: Parent directory imports (`from ..`) are not supported.

Standard Library (`std`)

The `std` package is automatically imported into every program. You don't need to write `from std` — it's always available.

```
# These functions are available without any import
print("hello")
n = length("world")
xs = range(5)
```

You can also explicitly import specific definitions from standard library packages:

```
from str import contains
```

`RY_HOME`

The standard library is installed at `$RY_HOME/share/std/`. The default value of `RY_HOME` is `~/.ry`.

```
export RY_HOME="$HOME/.ry"    # default
```

Search Path Priority

Package files are searched in the following order:

1. **Directory of the importing file** — The directory containing the file with the import statement is searched first.
 2. **`$RY_HOME/share`** — Standard library location. Falls back to `$RY_HOME/lib` for legacy installations.
 3. **Executable-relative `share/`** — Directories relative to the `ry` executable. Falls back to executable-relative `lib/` for legacy layouts.
 4. **`RY_PATH` environment variable** — If not found, directories specified in `RY_PATH` are searched in order.
-

`RY_PATH` Environment Variable

Multiple directories can be specified separated by colons.

```
export RY_PATH=/home/user/ry-libs:/usr/local/ry-libs
```

Once set, packages in the specified directories can be imported from anywhere.

Limitations

- `from` statements can only be written at the **top level** of a file. They cannot be placed inside functions or blocks.
- Importing the same package multiple times is automatically skipped (no duplicate imports).
- **Circular imports** (A imports B and B imports A) result in an error.

```
# Error example: a.ry and b.ry import each other
# a.ry: from b import foo
# b.ry: from a import bar <- circular import error
```

[<- Prev: Error Handling](#) | [Next: Concurrency ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Concurrency

[<- Prev: Packages](#) | [Next: Testing ->](#)

Ry provides three models for concurrent and parallel execution: **async/await** for lightweight I/O-bound tasks, **@parallel** for data-parallel loops, and **OS threads** for CPU-bound workloads with fine-grained control.

async/await

`Task<T>` is the runtime handle for concurrent work. Use `async` function to declare functions that return tasks, `await` inside another `async` function to wait for a task, and `block_on(task)` from synchronous context.

Defining Async Functions

```
async function add(a: int, b: int) -> int:
    return a + b
```

Calling an `async` function returns a `Task<T>` immediately — the work starts in the background:

```
t: Task<int> = add(20, 22)
```

Waiting for Results

From **synchronous** code, use `block_on()`:

```
print(block_on(t))           # 42
print(block_on(add(1, 2)))    # 3
```

From **async** code, use `await`:

```
async function double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2
```

```
print(block_on(double_add(3, 4)))    # 14
```

Composing Async Functions

You can chain `async` operations naturally:

```
async function fetch_score() -> int:
    return 42
```

```
async function process() -> str:
    score = await fetch_score()
    return f"Score: {score * 2}"
```

```
print(block_on(process())) # Score: 84
```

Why `async/await`? It lets you write concurrent code that reads like sequential code. The runtime schedules tasks efficiently on a thread pool — ideal for I/O-bound work where you spend most of the time waiting.

Common mistake: Using `await` in a non-async function causes a compile error. Use `block_on()` instead when calling from synchronous context.

@parallel

The `@parallel` directive parallelizes counted `for` loops across multiple CPU cores:

```
@parallel
for i in range(8):
    print(i)
```

Restrictions

`@parallel for` is designed for independent iterations. It rejects: - **break and continue** — there is no meaningful ordering across parallel iterations. - **Writes to outer mutable variables** — this would cause data races.

Only counted loops (`range(...)` or integer `..` ranges) are supported.

Why `@parallel`? It is the simplest way to parallelize independent work. No locks, no atomics — just add the directive and the runtime handles distribution.

OS Threads

For CPU-bound tasks or when you need fine-grained synchronization, use the `thread` module to create OS threads.

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add
```

Spawning Threads

`thread_spawn` takes a closure and starts a new OS thread:

```
counter = atomic_int_new(0)

t = thread_spawn():
    atomic_int_add(counter, 1)
)

thread_join(t)
print(atomic_int_load(counter)) # 1
```

Note: Captured variables are copied into the thread. For primitive types (`int`, `str`, etc.), this produces an independent copy. For opaque handle types (`AtomicInt`, `Lock`, etc.), the underlying resource is shared — which is exactly what you want for synchronization.

Shared State with Atomics

AtomicInt provides thread-safe integer operations without locks:

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add

counter = atomic_int_new(0)

t1 = thread_spawn():
    atomic_int_add(counter, 1)
)
t2 = thread_spawn():
    atomic_int_add(counter, 1)
)

thread_join(t1)
thread_join(t2)
print(atomic_int_load(counter))    # 2
```

Mutual Exclusion with Locks

When you need to protect a critical section (more than a single atomic operation), use a Lock:

```
from thread import thread_spawn, thread_join, lock_new, lock_acquire, lock_release
from thread import atomic_int_new, atomic_int_load, atomic_int_add

lock = lock_new()
counter = atomic_int_new(0)

t = thread_spawn():
    lock_acquire(lock)
    # Critical section: only one thread at a time
    atomic_int_add(counter, 1)
    lock_release(lock)
)

lock_acquire(lock)
atomic_int_add(counter, 1)
lock_release(lock)

thread_join(t)
print(atomic_int_load(counter))    # 2
```

Other Synchronization Primitives

The `thread` module also provides:

Primitive	Use Case
RWLock	Multiple readers or one writer (read-heavy workloads)
Semaphore	Limit concurrent access to a resource
Barrier	Wait until N threads reach the same point
AtomicBool	Thread-safe boolean flag

See [Thread Reference](#) for the full API.

Networking (TCP Sockets)

Ry provides TCP socket support through the `net` module. Network operations return `Result` types (from [Error Handling](#)) since connections can fail.

The `net` module provides the connection-management primitives, while `send`, `receive`, and `close` are language builtins (always in scope — no import needed) that operate on any stream-like handle:

Function	Source	Description
<code>bind(host, port)</code>	<code>net</code>	Allocate a listener. Returns <code>Result<TcpListener, Error></code>
<code>listen(listener, backlog)</code>	<code>net</code>	Start accepting connections. Returns <code>Result<Unit, Error></code>
<code>accept(listener)</code>	<code>net</code>	Wait for the next connection. Returns <code>Result<TcpStream, Error></code>
<code>connect(host, port)</code>	<code>net</code>	Open an outbound stream. Returns <code>Result<TcpStream, Error></code>
<code>send(stream, bytes)</code>	builtin	Send a <code>List<u8></code> . Returns <code>Result<int, Error></code>
<code>receive(stream, max)</code>	builtin	Read up to <code>max</code> bytes. Returns <code>Result<List<u8>, Error></code>
<code>close(handle)</code>	builtin	Release a <code>TcpListener</code> , <code>TcpStream</code> , or <code>TlsStream</code>

Only the `net` functions need to be imported. `send`, `receive`, and `close` are global builtins — writing `from net import send` produces a compile error.

An echo exchange between an async server and a synchronous client:

```
from net import bind, listen, accept, connect, listener_port
from io import to_bytes, bytes_to_str

async function echo_server(server: TcpListener) -> str:
    case accept(server):
        Ok(conn):
            case receive(conn, 4096):
                Ok(data):
                    case send(conn, data):
                        Ok(_):
                            ...
                        Err(e):
                            ...
                    Err(e):
                        ...
                close(conn)
            Err(e):
                ...
        close(server)
    return "done"

function run_client(port: int) -> str:
    case connect("127.0.0.1", port):
        Ok(conn):
            case send(conn, to_bytes("hello")):
                Ok(_):
```

```

        ...
        Err(e):
            ...
    case receive(conn, 4096):
        Ok(resp):
            case bytes_to_str(resp):
                Ok(msg):
                    close(conn)
                    return msg
                Err(e):
                    ...
            Err(e):
                ...
        close(conn)
        return "fail"
    Err(e):
        return "fail"

case bind("127.0.0.1", 0):
    Ok(server):
        case listen(server, 1):
            Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                print(run_client(port))    # hello
                block_on(t)
            Err(e):
                print(e.message)
    Err(e):
        print(e.message)

```

See [Network Reference](#) for the full TCP API, including TLS (`tls_connect`) and per-stream timeouts.

Choosing the Right Model

Feature	<code>async/await</code>	<code>@parallel</code>	<code>thread</code> module
Best for	I/O-bound tasks	Data-parallel loops	CPU-bound, fine-grained control
Overhead	Low (task scheduling)	Low (automatic)	Higher (OS thread creation)
Synchronization	Automatic via <code>await</code>	Not needed (independent iterations)	Manual (Lock, Semaphore, etc.)
Complexity	Medium	Low	High

Common Mistakes

1. **Using `await` outside `async` function:** Use `block_on()` from synchronous context instead.
2. **Writing to outer variables in `@parallel`:** This is rejected at compile time to prevent data races.
3. **Forgetting `thread_join`:** If the main program exits before a thread finishes, the thread's work may be lost.

4. **Sharing primitive variables between threads:** Primitives are copied into closures. Use `AtomicInt` or `Lock` for shared state.
-

Exercises

1. **async/await:** Write two `async function` functions —one that returns a first name and one that returns a last name. Write a third `async function` that `await`s both and returns the full name. Use `block_on()` to print the result.
 2. **Threads with atomics:** Spawn 5 threads that each add 10 to a shared `AtomicInt`. After joining all threads, verify the counter is 50.
-

[<- Prev: Packages](#) | [Next: Testing ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Testing

[<- Prev: Concurrency](#) | [Next: Building a Project ->](#)

Ry has a built-in RSpec-style test syntax using `@describe`, `@it`, and `expect`. For the full specification, see [Testing Reference](#).

Running Tests

```
ry test           # Auto-discover and run all *.test.ry files
ry test tests/spec # Run all *.test.ry files under a directory (recursive)
ry test tests/my_test.test.ry # Run a specific test file
ry test -p        # Run all tests in parallel (-p or --parallel)
```

The exit code is 0 if all tests pass, 1 if any test fails.

When run without arguments, `ry test` searches for `package.toml` to find the project root, then recursively discovers all `*.test.ry` files.

Writing Tests

Attach `@it` to a named function to declare a test case, and wrap a group of related tests in a function annotated with `@describe`.

```
@it("should add integers")
function test_add():
  expect(1 + 2).to_eq(3)

@describe("Calculator")
function calculator_tests():
  @it("should subtract integers")
  function test_sub():
    expect(5 - 3).to_eq(2)

  @it("should check booleans")
```

```
function test_bool():
  expect(3 > 1).to_be_true()
```

- `@it` takes a description string. The decorated function becomes the test body
- `@describe` groups the inner `@it` functions defined in its body. Groups may be nested; output is indented proportionally to nesting depth
- Variables declared directly in the body of a `@describe` function act as **shared setup** and are captured by every inner `@it` function

```
@describe("shared setup")
function shared_setup_tests():
  base = 100
  offset = 5

  @it("should use base value")
  function test_base():
    expect(base).to_eq(100)

  @it("should use base and offset")
  function test_combined():
    expect(base + offset).to_eq(105)
```

- `expect`, `mock`, and `verify` are only available with `ry test` (compile error with normal `ry` execution)

Legacy lambda form: `describe("name", (): ...)` and `it("desc", (): ...)` with a lambda argument still parse but are **deprecated**. Prefer the directive form on named functions for new code.

Matchers

Matcher	Description	Supported Types
<code>to_eq(expected)</code>	Equality comparison	int, float, bool, str
<code>to_not_eq(expected)</code>	Asserts not equal	int, float, bool, str
<code>to_be_true()</code>	Asserts true	bool
<code>to_be_false()</code>	Asserts false	bool
<code>to_be_none()</code>	Asserts None	Option
<code>to_be_some()</code>	Asserts Option is Some	Option
<code>to_be_ok()</code>	Asserts Result is Ok	Result
<code>to_be_err()</code>	Asserts Result is Err	Result
<code>to_contain(val)</code>	Asserts container includes value	List, Set, Map, str
<code>to_not_contain(val)</code>	Asserts container does not include value	List, Set, Map, str
<code>to_be_greater_than(v)</code>	Asserts actual > v	int, float
<code>to_be_less_than(v)</code>	Asserts actual < v	int, float
<code>to_be_greater_than_or_eq(v)</code>	Asserts actual >= v	int, float
<code>to_be_less_than_or_eq(v)</code>	Asserts actual <= v	int, float
<code>to_have_length(n)</code>	Asserts length equals n	List, Set, Map, str
<code>to_be_empty()</code>	Asserts length is 0	List, Set, Map, str
<code>to_start_with(prefix)</code>	Asserts string starts with prefix	str
<code>to_end_with(suffix)</code>	Asserts string ends with suffix	str

fail

`fail()` immediately marks the current test as failed.

```

@it("should handle error")
function test_should_handle_error():
  result = find_user(-1)
  case result:
    Ok(v):
      fail("expected error")
    Err(e):
      expect(e.message).to_eq("not found")

```

- `fail()` — marks the test as failed with a generic message
 - `fail(message)` — marks the test as failed with a custom message
 - Only available in `ry test` mode
-

Output Format

Calculator

```

+ should add integers
+ should subtract integers
- should check booleans
line 10: expected true, got false

```

2 passed, 1 failed

+ indicates pass (green), - indicates failure (red). Nested `@describe` groups indent their inner tests proportionally to the nesting depth:

```

outer group
  inner group
    + should pass deeply nested test

```

Mocking

`mock(fn_name, replacement)`

Replaces a function with a mock implementation for the current `it` block. The mock is automatically restored when the `it` block ends. The original function's `require` and `ensure` contracts still run for mocked calls.

```

function fetch_data() -> str:
  return "real data"

@describe("mocking")
function mocking_tests():
  @it("should replace function")
  function test_replace():
    mock(fetch_data, () => "fake")
    expect(fetch_data()).to_eq("fake")

  @it("should auto-restore after it block")
  function test_restore():
    expect(fetch_data()).to_eq("real data")

```

`verify(fn_name)`

Returns the number of times a mocked function was called.

```

@describe("verify")
function verify_tests():
  @it("should count calls")
  function test_count():
    mock(fetch_data, () => "fake")
    fetch_data()
    fetch_data()
    expect(verify(fetch_data)).to_eq(2)

```

Parameterized Tests

Combine `@each` with `@it` on a named function to run the same test with multiple inputs:

```

@each([(1, 2, 3), (0, 0, 0), (-1, 1, 0)])
@it("should add {0} + {1} = {2}")
function test_add_each(a: int, b: int, expected: int):
  expect(a + b).to_eq(expected)

```

Each tuple becomes a separate test case. `{0}`, `{1}`, etc. in the description are replaced with actual values. The function's parameter list must match the tuple arity.

Property-Based Tests

Combine `@property` with `@it` on a named function to test with randomly generated inputs:

```

@property(count=100)
@it("should verify addition is commutative")
function test_add_commutative(a: int, b: int):
  expect(a + b).to_eq(b + a)

```

The test runs `count` times with random values. Ry infers the generator from each parameter's type annotation. On failure, the counterexample is printed.

Testing with Contracts

Contracts (from [Error Handling](#)) work together with mocking: the original function's `require` and `ensure` contracts **still run** for mocked calls. This means contracts act as implicit test assertions.

```

function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
  ensure v:
    v > balance
  return balance + amount

@describe("deposit")
function deposit_tests():
  @it("should enforce contracts even when mocked")
  function test_contract():
    mock(deposit, (amount: int, balance: int) => balance + amount)
    expect(deposit(10, 100)).to_eq(110)
    # deposit(-1, 100) would terminate with "require failed"

```

Why this matters: You can mock implementation details while keeping the contract safety net. If a mock violates a postcondition, the test catches it immediately.

Limitations

- Nesting is only supported with `@describe` on named functions. The legacy lambda form `describe("name", (): ...)` cannot be nested
 - `before_each` / `after_each` are not supported — use shared setup variables declared in a `@describe` function body instead
 - Overloaded and `@native` functions cannot be mocked
-

Exercises

1. **Basic testing:** Write a `describe` block with tests for a `max(a: int, b: int) -> int` function, covering equal values, positive numbers, and negative numbers.
 2. **Mocking:** Write a function `fetch_temperature() -> int` that returns a value. Mock it in a test to return a fixed value and use `verify` to check it was called exactly once.
 3. **Parameterized tests:** Use `@each` to test a `is_even(n: int) -> bool` function with inputs [(2, true), (3, false), (0, true), (-4, true)].
-

[<- Prev: Concurrency](#) | [Next: Building a Project ->](#)

[English](#) | [日本語](#) | [繁體中文](#)

Building a Project

[<- Prev: Testing](#)

In this tutorial, you will build a **task tracker** — a small application that manages an in-memory to-do list. This project ties together most of the language features you have learned:

- **Records and ADT enums** (data modeling)
 - **Collections and iterators** (filtering and transforming tasks)
 - **Error handling** (Result, Option, contracts)
 - **F-strings and UFCS** (readable output and chaining)
 - **Functions with default arguments** (flexible API)
 - **Modules** (organizing code across files)
 - **Testing** (verifying behavior)
-

Project Setup

Create a new project:

```
ry new task-tracker
cd task-tracker
```

This gives you:

```
task-tracker/
package.toml
```

```
src/  
  main.ry
```

Step 1: Define the Data Model

Create `src/model.ry` with the task record and status enum:

```
enum Status:  
  Todo  
  InProgress  
  Done  
  
record Task:  
  id: int  
  title: str  
  status: Status  
  invariant:  
    length(title) > 0  
  
function create_task(id: int, title: str, status: Status = Status::Todo) -> Task:  
  require:  
    length(title) > 0  
  return Task(id, title, status)
```

This uses several features at once: - **ADT enum** for `Status` (from [Records and Enums](#)) - **Record invariant** to ensure the title is never empty (from [Error Handling](#)) - **Default argument** so `status` defaults to `Todo` (from [Functions](#)) - **Contract** (`require`) as a safety net on construction (from [Error Handling](#))

Step 2: Task Operations

Add functions to `src/model.ry` for working with task lists:

```
function add_task(tasks: List<Task>, title: str) -> List<Task>:  
  id = length(tasks) + 1  
  task = create_task(id, title)  
  append(tasks, task)  
  return tasks  
  
function find_task(tasks: List<Task>, id: int) -> Option<Task>:  
  for t in tasks:  
    if t.id == id:  
      return Some(t)  
  return None  
  
function complete_task(tasks: List<Task>, id: int) -> Result<List<Task>, Error>:  
  case find_task(tasks, id):  
    Some(t):  
      t.status = Status::Done  
      return Ok(tasks)  
    None:  
      return Err(Error(f"task {id} not found"))  
  
function pending_tasks(tasks: List<Task>) -> List<Task>:
```

```

return tasks
    .iter()
    .filter((t: Task) => t.status == Status::Todo)
    .to_list()

```

Notice the patterns: - **Option** for `find_task` — a task may not exist (from [Error Handling](#)) - **Result** for `complete_task` — completing a nonexistent task is an error - **Iterator pipeline** with UFCS chaining in `pending_tasks` (from [Collections](#) and [Functions](#)) - **F-string** for the error message (from [Variables and Types](#))

Step 3: Display

Add a display function to `src/model.ry`:

```

function format_task(t: Task) -> str:
    marker = "[ ]"
    case t.status:
        Status::Todo:
            marker = "[ ]"
        Status::InProgress:
            marker = "[~]"
        Status::Done:
            marker = "[x]"
    return f"{marker} {t.id}. {t.title}"

function print_tasks(tasks: List<Task>):
    if is_empty(tasks):
        print("No tasks.")
        return
    for t in tasks:
        print(format_task(t))

```

Step 4: Main Program

Edit `src/main.ry`:

```

from model import create_task, add_task, complete_task, pending_tasks, print_tasks, Status, Task

tasks: List<Task> = []

# Add some tasks
tasks = add_task(tasks, "Buy groceries")
tasks = add_task(tasks, "Write documentation")
tasks = add_task(tasks, "Review pull request")

print("All tasks:")
print_tasks(tasks)

# Complete a task
case complete_task(tasks, 1):
    Ok(updated):
        tasks = updated
    Err(e):
        print(f"Error: {e}")

```

```
print("\nPending tasks:")
print_tasks(pending_tasks(tasks))
```

Run it:

```
ry src/main.ry
```

Expected output:

```
All tasks:
[ ] 1. Buy groceries
[ ] 2. Write documentation
[ ] 3. Review pull request
```

```
Pending tasks:
[ ] 2. Write documentation
[ ] 3. Review pull request
```

Step 5: Write Tests

Create tests/model.test.ry:

```
from model import create_task, add_task, find_task, complete_task, pending_tasks, Status, Task
```

```
describe("Task model", ():
    it("creates a task with default status", ():
        t = create_task(1, "Test task")
        expect(t.status == Status::Todo).to_be_true()
    )

    it("adds tasks to a list", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "First")
        tasks = add_task(tasks, "Second")
        expect(length(tasks)).to_eq(2)
    )

    it("finds a task by id", ():
        tasks: List<Task> = []
        tasks = add_task(tasks, "Target")
        case find_task(tasks, 1):
            Some(t):
                expect(t.title).to_eq("Target")
            None:
                fail("expected to find task")
    )

    it("returns None for missing task", ():
        tasks: List<Task> = []
        expect(find_task(tasks, 99)).to_be_none()
    )

    it("completes a task", ():
        tasks: List<Task> = []
```

```

tasks = add_task(tasks, "Do it")
case complete_task(tasks, 1):
    Ok(updated):
        remaining = pending_tasks(updated)
        expect(is_empty(remaining)).to_be_true()
    Err(e):
        fail("unexpected error")
)

it("returns error for invalid id", ():
    tasks: List<Task> = []
    case complete_task(tasks, 999):
        Ok(_):
            fail("expected error")
        Err(e):
            expect(e.message != "").to_be_true()
)
)

```

Run the tests:

```
ry test
```

Next Steps

Here are some ways to extend this project:

- **Add due dates** using `Option<str>` for an optional due date field
- **Persist to JSON** using the `json` module to save/load tasks from a file
- **Add priorities** by extending `Status` into an ADT with priority levels
- **Concurrency** — process a batch of tasks in parallel using `@parallel`

You now have a solid foundation in Ry. Explore the [Reference](#) documentation for the full language specification.

[<- Prev: Testing](#)